



Sistema de Gestão de Clínica Veterinária

Relatório técnico – 3ª Iteração

- Helder Souza Nº 1231982
- Rui Pinto Nº 1230906
- Rodrigo Martins Nº 1241122

Índice

1. Introdução	3
2. Objetivo do Projeto	4
3. Arquitetura do Sistema.....	5
3.1 Organização em Camadas	5
3.2 Responsabilidades das Classes Principais.....	6
3.3 Fluxo Geral da Aplicação.....	7
4. Padrões e Estruturas Utilizadas	7
4.1 MVC Simplificado	7
4.2 Service Layer	8
4.3 DTO	8
4.4 Mapper.....	9
4.5 Repository Pattern	10
4.6 Exceptions	10
4.7 Relações por Apontadores no Model	11
5. Modelo de Domínio.....	12
5.1 Entidades Principais	13
5.2 Relações entre Entidades	13
5.3 Uso de Apontadores no Model.....	14
5.4 Uso de IDs na Interface e nos DTOs	15
6. Persistência de Dados	15
6.1 Repository Binário	16
6.2 Ficheiro clinic.dat	17
6.3 Save e Load.....	17
6.4 Reconstrução de Apontadores	18
7. Funcionalidades Implementadas.....	19
7.1 Gestão de Animais	19
7.2 Gestão de Veterinários.....	20
7.3 Gestão de Serviços.....	20

7.4 Gestão de Prescrições	21
7.5 Consultas e Listagens.....	21
8. Tratamento de Erros e Validações	22
8.1 InvalidDataException.....	23
8.2 NoDataException	23
8.3 DuplicatedDataException	24
8.4 DataConsistencyException	24
9. Testes	25
9.1 Estrutura do Projeto de Testes	25
9.2 Testes ao Model e Containers.....	26
9.3 Testes aos Services	27
9.4 Testes ao Repository.....	27
9.5 Resultado dos Testes	28
10. Diagramas.....	28
10.1 Diagrama de Arquitetura	28
10.2 Diagramas de Classes	29
10.3 Diagrama do Repository	30
10.4 Diagramas de Casos de Uso	31
11. Limitações e Trabalho Futuro	32
12. Conclusão.....	33
13. Referências	34
Figura 1-Exemplo do fluxo entre Controller, Service, Repository e View	8
Figura 2-Validação de relações no ServiceService.....	8
Figura 3-Exemplo de DTO de entrada	9
Figura 4-Conversão de Prescription para DTO.....	9
Figura 5-Interface IClinicRepository.....	10
Figura 6-Inicialização do Repository no Controller	10
Figura 7-Exemplo de lançamento de NoDataException	11
Figura 8-Relações por apontadores na classe Service.....	11
Figura 9-Reconstrução de apontadores a partir de IDs	12

Figura 10-Modelo de domínio da aplicação	12
Figura 11-Representação de apontadores no Model	14
Figura 12-Interface do Repository	16
Figura 13-Acesso à Clinic através do Repository	16
Figura 14-Abertura do ficheiro binário para escrita	17
Figura 15-Abertura do ficheiro binário para leitura.....	17
Figura 16-Escrita da quantidade de objetos no ficheiro binário	18
Figura 17-Escrita de uma string no ficheiro binário	18
Figura 18-Reconstrução de apontadores no carregamento dos dados	19
Figura 19-Exemplo de validação com InvalidDataException	23
Figura 20-Exemplo de utilização de NoDataException	23
Figura 21-Exemplo de utilização de DuplicatedDataException	24
Figura 22-Exemplo de utilização de DataConsistencyException.....	24
Figura 23-Execução dos testes com Google Test.....	26
Figura 24-Resultado dos testes automáticos	28
Figura 25-Diagrama de arquitetura da aplicação.....	29
Figura 26-Diagrama do Repository	30
Figura 27-Diagrama geral de casos de uso.....	32

1. Introdução

Este relatório apresenta o desenvolvimento de uma aplicação de consola em C++, com o objetivo de gerir uma clínica veterinária. O projeto foi realizado no âmbito da unidade curricular de Fundamentos de Desenvolvimento de Software e teve como base o exemplo *School Application*, aplicação fornecida como referência para a estrutura do projeto.

A aplicação permite gerir informação relativa a animais, veterinários, serviços e prescrições. Ao longo do desenvolvimento, foi aplicada uma arquitetura organizada em camadas, separando responsabilidades entre interface, controlo, lógica da aplicação, modelo de domínio, persistência de dados e testes.

O sistema foi desenvolvido com conceitos da Programação Orientada a Objetos, representando as entidades principais através de classes e estabelecendo relações entre objetos. Foi também implementada a funcionalidade de persistência em ficheiro binário, o tratamento de erros através de exceptions próprias e testes automáticos com Google Test.

2. Objetivo do Projeto

O objetivo principal deste projeto é desenvolver uma aplicação que permite gerir, de forma simples e estruturada, as principais funções de uma clínica veterinária.

A aplicação permite ao utilizador:

- registar, editar e remover animais;
- registar, editar e remover veterinários;
- criar, editar e remover serviços associados a animais e veterinários;
- emitir, editar e remover prescrições a animais;
- consultar animais por ID;
- listar animais, veterinários, serviços e prescrições;
- consultar serviços associados a um veterinário;
- consultar prescrições associadas a um animal;
- guardar e carregar os dados entre execuções da aplicação.

Além destas funcionalidades, outro objetivo foi aplicar conceitos de desenvolvimento de software estudados durante a unidade curricular, nomeadamente:

- organização do código em camadas;
- uso de classes, objetos e relações entre entidades;
- uso de DTOs e Mappers;
- aplicação do padrão Repository para persistência;
- tratamento de erros com exceptions;
- criação de testes automáticos;
- aproximação à estrutura usada em *School Application*.

Desta forma, o projeto não está limitado apenas à implementação das funcionalidades da clínica, mas também serve como exercício de aplicação prática de conceitos de Programação Orientada a Objetos e organização de software.

3. Arquitetura do Sistema

A aplicação foi desenvolvida à volta de uma arquitetura organizada em camadas, inspirada na estrutura da aplicação do exemplo *School Application*. Esta organização permite separar responsabilidades, tornar o código mais fácil de compreender e facilitar futuras alterações ou melhorias.

A estrutura principal da aplicação divide-se nas seguintes camadas:

- **Views**
- **Controller**
- **Services**
- **DTOs**
- **Mappers**
- **Model**
- **Containers**
- **Repository**
- **Exceptions**

Cada camada tem uma responsabilidade específica dentro do sistema.

3.1 Organização em Camadas

A camada **View** é a camada responsável pela interação do utilizador através da consola. É nesta camada que são apresentados os menus, pedidos os dados ao utilizador e mostrados os resultados das operações.

A camada **Controller** é responsável por coordenar o funcionamento da aplicação. O Controller apresenta os menus principais e chama os serviços correspondentes dependendo das opções escolhidas pelo utilizador.

A camada **Services** contém a lógica principal da aplicação. É nesta camada que são feitas as validações, consultas aos containers e criação de novos objetos do domínio.

Os **DTOs** são responsáveis por transportar dados entre a interface e os serviços. Por exemplo, quando o utilizador cria um novo serviço, os dados introduzidos são guardados num `ServiceInDTO` antes de serem tratados pelo `ServiceService`.

Os **Mappers** são responsáveis por converter objetos do Model em DTOs de saída. Desta forma, as Views recebem dados simples para apresentar ao utilizador, sem depender diretamente das classes internas do domínio.

A camada **Model** contém as entidades principais da aplicação, como Animal, Veterinarian, Service e Prescription.

Os **Containers** gerem coleções de objetos do Model. Cada entidade principal tem o seu container, responsável por guardar, procurar e gerir os objetos correspondentes.

A camada **Repository** é responsável pela persistência dos dados. Neste projeto, foi implementado o ClinicRepositoryBinary, que guarda e carrega os dados da aplicação através num ficheiro binário.

Por fim, as **Exceptions** representam erros específicos da aplicação, como dados inválidos, objetos inexistentes ou inconsistências entre entidades.

3.2 Responsabilidades das Classes Principais

A classe Clinic funciona como classe central do domínio. Esta classe agrega os containers principais da aplicação:

- AnimalContainer
- VeterinarianContainer
- ServiceContainer
- PrescriptionContainer

Desta forma, a Clinic representa o estado geral da aplicação em memória.

O Controller é responsável por gerir o fluxo da aplicação. Ele comunica com as Views para recolher dados e apresentar informação, e chama os Services para executar as operações pretendidas.

Os Services, como AnimalService, VeterinarianService, ServiceService e PrescriptionService, contêm a lógica associada a cada área da aplicação. Por exemplo, o ServiceService valida se o animal e o veterinário existem antes de criar um novo serviço.

As entidades do Model representam os conceitos principais do domínio da clínica veterinária. Por exemplo, um Service está associado a um Animal e a um Veterinarian, enquanto uma Prescription também está associada a estas duas entidades.

A persistência é tratada pelo `ClinicRepositoryBinary`, que guarda e carrega o estado da `Clinic` através de um ficheiro binário.

3.3 Fluxo Geral da Aplicação

Na implementação real da aplicação, o `Controller` é o ponto central de coordenação. Ele apresenta os menus, chama as `Views` para recolher ou mostrar informação, chama os `Services` para executar a lógica da aplicação e usa o `Repository` para carregar e guardar os dados.

De forma geral, o fluxo da aplicação começa no utilizador, passa pela `View`, é coordenado pelo `Controller`, processado pelos `Services` e aplicado sobre a `Clinic`, os `Containers` e o `Model`.

A `Clinic` funciona como objeto central do domínio, contendo os containers de animais, veterinários, serviços e prescrições. Os `Services` trabalham sobre essa `Clinic`, enquanto o `Repository` é responsável por guardar e carregar o seu estado.

Quando a aplicação inicia, o `Controller` chama `repository.load()`. Sempre que há alterações aos dados, como criar um animal, veterinário, serviço ou prescrição, o `Controller` chama `repository.save()`. Assim, a aplicação mantém os dados em memória através da `Clinic` e guarda-os no ficheiro binário através do `ClinicRepositoryBinary`.

A organização geral da aplicação em camadas é apresentada no diagrama de arquitetura da secção 10.1.

4. Padrões e Estruturas Utilizadas

Durante o desenvolvimento da aplicação foram usadas várias estruturas e padrões de organização de código, inspirados no exemplo *School Application*. Estes padrões ajudaram a separar responsabilidades e a tornar o projeto mais organizado e fácil de manter.

4.1 MVC Simplificado

A aplicação segue uma estrutura semelhante ao padrão MVC, adaptada a uma aplicação de consola. A camada `View` é responsável pela interação com o utilizador,

o Controller coordena o fluxo da aplicação e o Model representa as entidades principais do domínio.

Exemplo simplificado do fluxo no Controller:

```
animalService.addAnimal(dto);  
repository.save();  
animalView.showAnimalCreated();
```

Figura 1-Exemplo do fluxo entre Controller, Service, Repository e View

Neste exemplo, a View recolhe os dados, o Controller coordena a operação e o Service executa a lógica necessária para criar o animal.

4.2 Service Layer

Foi usada uma camada de Services para concentrar a lógica principal da aplicação. Desta forma, as validações e operações sobre os dados não ficam diretamente nas Views nem no Controller.

Exemplo de validação no ServiceService:

```
Animal* animal = clinic.getAnimalContainer().get(dto.animalId);  
Veterinarian* veterinarian = clinic.getVeterinarianContainer().get(dto.veterinarianId);  
  
if (animal == nullptr || veterinarian == nullptr) {  
    throw DataConsistencyException("Animal ou Veterinário não existe.");  
}
```

Figura 2-Validação de relações no ServiceService

Este padrão permite que a lógica da aplicação fique centralizada nos Services, tornando o código mais organizado.

4.3 DTO

Os DTOs foram usados para transportar dados entre a interface e os Services. Isto evita que a View trabalhe diretamente com os objetos internos do Model.

Exemplo de DTO usado ao criar um serviço:

```

struct ServiceInDTO {
    std::string type;
    float cost;
    std::string date;
    std::string time;
    int animalId;
    int veterinarianId;
    bool requiresVeterinarianSpecialty{};
    std::string requiredVeterinarianSpecialty;
};

```

Figura 3-Exemplo de DTO de entrada

Neste caso, o utilizador introduz os IDs do animal e do veterinário na consola. Depois, o ServiceService usa esses IDs para encontrar os objetos reais e criar a relação no Model.

4.4 Mapper

Os Mappers foram usados para converter objetos do Model em DTOs de saída. Assim, a View recebe dados simples para apresentar ao utilizador.

Exemplo de conversão no PrescriptionMapper:

```

PrescriptionOutDTO PrescriptionMapper::toDTO(const Prescription& prescription) {
    PrescriptionOutDTO dto{};

    dto.id = prescription.getId();
    dto.medication = prescription.getMedication();
    dto.quantity = prescription.getQuantity();
    dto.duration = prescription.getDuration();
    dto.animalId = prescription.getAnimal()->getId();
    dto.veterinarianId = prescription.getVeterinarian()->getId();

    return dto;
}

```

Figura 4-Conversão de Prescription para DTO

Desta forma, os objetos internos do Model não precisam de ser expostos diretamente à camada de apresentação.

4.5 Repository Pattern

Foi usado o padrão Repository para separar a lógica da aplicação da forma como os dados são guardados. O Repository é responsável por guardar e carregar a Clinic.

Interface do Repository:

```
class IClinicRepository {
public:
    virtual ~IClinicRepository() = default;

    virtual Clinic& getClinic() = 0;
    virtual void save() = 0;
};
```

Figura 5-Interface IClinicRepository

No Controller, a aplicação usa o repository para aceder à Clinic e guardar os dados:

```
Controller::Controller()
: repository( fileName: "clinic.dat"),
  animalService( [&] repository.getClinic()),
  veterinarianService( [&] repository.getClinic()),
  serviceService( [&] repository.getClinic()),
  prescriptionService( [&] repository.getClinic()) {
    try {
        repository.load();
    }
```

Figura 6-Inicialização do Repository no Controller

Neste projeto foi implementado o ClinicRepositoryBinary, que guarda e carrega os dados através do ficheiro binário clinic.dat.

4.6 Exceptions

Foram criadas exceptions próprias para representar erros específicos da aplicação, como dados inválidos, objetos inexistentes ou relações inválidas entre entidades.

Exemplo:

```

if (animal == nullptr) {
    throw NoDataException("Animal não encontrado.");
}

```

Figura 7-Exemplo de lançamento de NoDataException

As principais exceptions usadas são:

- InvalidDataException
- NoDataException
- DuplicatedDataException
- DataConsistencyException

O uso de exceptions torna o tratamento de erros mais claro do que devolver valores especiais, como -1 ou nullptr, em todas as situações.

4.7 Relações por Apontadores no Model

Embora não seja exatamente um padrão de arquitetura, uma decisão importante foi representar as relações no Model através de apontadores para objetos.

Exemplo na classe Service:

```

class Service {
private:
    int id{};
    std::string type{};
    float cost{};
    Date date{};
    Time time{};
    Animal* animal{};
    Veterinarian* veterinarian{};
}

```

Figura 8-Relações por apontadores na classe Service

Assim, um Service está diretamente associado a um Animal e a um Veterinarian, em vez de guardar apenas os seus IDs.

Na persistência binária, os apontadores não são guardados diretamente, porque endereços de memória não são válidos entre execuções da aplicação. Por isso, no ficheiro são guardados os IDs das entidades associadas e, no load(), esses IDs são usados para reconstruir os apontadores.

Exemplo:

```
Animal* animal = clinic.getAnimalContainer().get(animalId);
Veterinarian* veterinarian = clinic.getVeterinarianContainer().get(veterinarianId);
```

Figura 9-Reconstrução de apontadores a partir de IDs

Desta forma, em memória as relações continuam orientadas a objetos, enquanto no ficheiro são guardados dados simples e seguros.

5. Modelo de Domínio

O modelo de domínio representa os principais conceitos da clínica veterinária e as relações entre eles. Este diagrama tem como objetivo mostrar uma visão conceptual do problema, sem incluir detalhes internos da implementação, como containers, DTOs, mappers ou repository.

As entidades principais representadas são Clínica, Animal, Veterinário, Serviço e Prescrição. Estas entidades correspondem aos conceitos fundamentais necessários para gerir os dados da clínica.

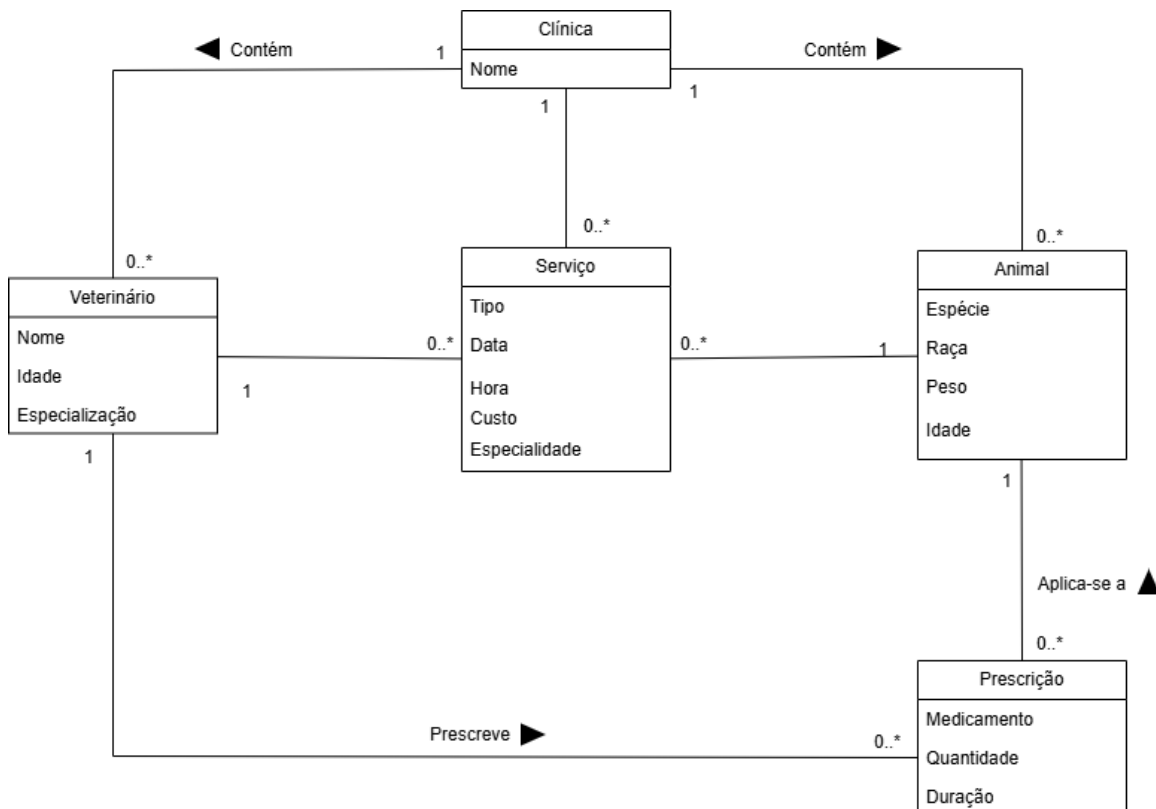


Figura 10-Modelo de domínio da aplicação

5.1 Entidades Principais

As entidades principais do domínio são Clínica, Animal, Veterinário, Serviço e Prescrição.

A Clínica representa a entidade principal do sistema, sendo responsável por agregar os dados relacionados com a gestão da clínica veterinária.

O Animal representa um animal registado na clínica. No modelo conceptual, contém informação como espécie, raça, peso e idade.

O Veterinário representa um profissional da clínica. Cada veterinário contém informação como nome, idade e especialização.

O Serviço representa um serviço realizado ou marcado na clínica. Este serviço está associado a um animal e a um veterinário, contendo informação como tipo, data, hora, custo e especialidade necessária.

A Prescrição representa uma prescrição emitida por um veterinário para um animal. Esta entidade contém informação como medicamento, quantidade e duração.

Entidade	Responsabilidade
Clínica	Representa a clínica veterinária
Animal	Representa um animal registado na clínica
Veterinário	Representa um profissional da clínica
Serviço	Representa um serviço realizado a um animal por um veterinário
Prescrição	Representa uma prescrição emitida para um animal

5.2 Relações entre Entidades

A Clínica contém os principais elementos do domínio, nomeadamente animais, veterinários e serviços. Uma clínica pode ter vários animais registados, vários veterinários e vários serviços associados.

Um Serviço está relacionado com um Animal e com um Veterinário. Isto significa que cada serviço é realizado por um veterinário e aplicado a um animal. Da mesma

forma, um animal pode receber vários serviços ao longo do tempo, e um veterinário pode realizar vários serviços.

A Prescrição também está relacionada com Animal e Veterinário. Uma prescrição aplica-se a um animal e é emitida por um veterinário. Assim, é possível identificar qual o animal a que a prescrição se destina e qual o veterinário responsável.

De forma resumida, as relações principais são:

- ❖ Clínica contém Animais
- ❖ Clínica contém Veterinários
- ❖ Clínica contém Serviços

- ❖ Veterinário realiza Serviços
- ❖ Animal recebe Serviços

- ❖ Veterinário prescreve Prescrições
- ❖ Prescrição aplica-se a Animal

5.3 Uso de Apontadores no Model

Apesar de o diagrama de domínio ser conceptual, na implementação real as relações entre entidades foram representadas através de apontadores no Model.

Por exemplo, um Service está associado diretamente a um Animal e a um Veterinarian. O mesmo acontece com uma Prescription, que também está associada a um animal e a um veterinário.

Isto significa que, no código, estas relações não são representadas apenas por IDs. Em memória, o sistema trabalha com ligações reais entre objetos.

Exemplo conceptual da implementação:

```
Animal* animal{};  
  
Veterinarian* veterinarian{};
```

Figura 11-Representação de apontadores no Model

Esta decisão permite que o modelo em memória seja mais orientado a objetos, uma vez que um serviço ou prescrição está diretamente ligado às entidades correspondentes.

5.4 Uso de IDs na Interface e nos DTOs

Embora o Model use apontadores para representar as relações entre objetos, a interface e os DTOs usam IDs.

Isto acontece porque o utilizador interage com a aplicação através da consola. Ao criar um serviço ou uma prescrição, o utilizador introduz o ID do animal e o ID do veterinário. Esses IDs são transportados pelos DTOs até aos Services.

Depois, os Services usam esses IDs para procurar os objetos reais nos dados da clínica. Se o animal e o veterinário existirem, o serviço ou a prescrição é criado com apontadores para esses objetos.

Assim, existe uma separação clara entre a forma como os dados são introduzidos pelo utilizador e a forma como as relações são guardadas no Model.

Camada	Representação da relação
Interface	O utilizador introduz IDs
DTOs	Transportam animalId e veterinarianId
Services	Procuram os objetos correspondentes
Model	Guarda apontadores para Animal e Veterinarian
Repository	Guarda IDs no ficheiro binário

6. Persistência de Dados

A persistência de dados foi implementada através da camada Repository. O objetivo desta camada é separar a lógica da aplicação da forma como os dados são guardados e carregados.

Neste projeto, a persistência é feita em ficheiro binário através da classe `ClinicRepositoryBinary`. Esta classe contém uma instância de `Clinic` e é responsável por guardar e carregar o estado da aplicação.

Quando a aplicação é iniciada, o `Controller` chama o método `load()` do `repository`. Este método tenta carregar os dados guardados anteriormente. Quando são feitas alterações aos dados, como registar um animal, veterinário, serviço ou prescrição, o `Controller` chama o método `save()` para guardar novamente o estado atual da aplicação.

6.1 Repository Binário

A classe principal usada para persistência é o `ClinicRepositoryBinary`. Esta classe implementa a interface `IClinicRepository`.

A interface define as operações principais do `repository`:

```
class IClinicRepository {
public:
    virtual ~IClinicRepository() = default;

    virtual Clinic& getClinic() = 0;
    virtual void save() = 0;
};
```

Figura 12-Interface do Repository

O `ClinicRepositoryBinary` guarda a informação da aplicação num ficheiro binário e disponibiliza a `Clinic` aos `Services` através do método `getClinic()`.

De forma simplificada:

```
Clinic& ClinicRepositoryBinary::getClinic() {
    return clinic;
}
```

Figura 13-Acesso à Clinic através do Repository

Isto permite que os `Services` continuem a trabalhar sobre a `Clinic`, enquanto a responsabilidade de guardar e carregar os dados fica separada no `Repository`.

6.2 Ficheiro clinic.dat

Os dados são guardados no ficheiro:

clinic.dat

Este ficheiro é criado automaticamente quando o método `save()` é chamado pela primeira vez. Normalmente, o ficheiro fica na pasta de execução da aplicação, por exemplo dentro da pasta de build criada pelo CLion.

O ficheiro é aberto em modo binário:

```
std::ofstream file(fileName, mode: std::ios::binary);
```

Figura 14-Abertura do ficheiro binário para escrita

E carregado também em modo binário:

```
std::ifstream file(fileName, mode: std::ios::binary);
```

Figura 15-Abertura do ficheiro binário para leitura

Se o ficheiro ainda não existir, o método `load()` termina sem erro e a aplicação inicia com uma Clinic vazia. Isto permite que a aplicação funcione normalmente na primeira execução.

6.3 Save e Load

O método `save()` guarda os dados existentes na Clinic para o ficheiro binário. O método `load()` faz o processo inverso, lendo o ficheiro e reconstruindo os objetos em memória.

A ordem usada no ficheiro é importante. Os dados são guardados e carregados pela seguinte ordem:

1. Animais
2. Veterinários
3. Serviços
4. Prescrições

Antes de guardar cada conjunto de objetos, é guardado o número de elementos existentes. Por exemplo, antes de guardar os animais, é guardada a quantidade de animais:

```
int animalCount = animals.size();
file.write(reinterpret_cast<const char*>(&animalCount), n: sizeof(animalCount));
```

Figura 16-Escrita da quantidade de objetos no ficheiro binário

Depois, o load() lê esse número e sabe quantos objetos precisa de reconstruir:

```
int animalCount;
file.read(reinterpret_cast<char*>(&animalCount), sizeof(animalCount));
```

As strings são guardadas de forma especial, porque têm tamanho variável. Para isso, é guardado primeiro o tamanho da string e depois o seu conteúdo.

Exemplo simplificado:

```
int size = static_cast<int>(value.size());
file.write(reinterpret_cast<const char*>(&size), n: sizeof(size));
file.write(s: value.c_str(), size);
```

Figura 17-Escrita de uma string no ficheiro binário

No carregamento, é lido primeiro o tamanho e depois o conteúdo da string.

Esta abordagem evita guardar diretamente a memória bruta dos objetos, o que seria perigoso, uma vez que os objetos têm std::string e apontadores.

6.4 Reconstrução de Apontadores

Uma parte importante da persistência está relacionada com as entidades que possuem relações com outras entidades.

No Model, as classes Service e Prescription guardam apontadores para Animal e Veterinarian:

```
Animal* animal;
Veterinarian* veterinarian;
```

Estes apontadores não podem ser guardados diretamente no ficheiro, porque representam endereços de memória válidos apenas durante uma execução da aplicação. Quando a aplicação é fechada e aberta novamente, esses endereços deixam de ser válidos.

Por isso, no ficheiro binário são guardados os IDs do animal e do veterinário associados. Por exemplo, ao guardar um serviço, são guardados:

serviceId
type
cost
date
time
animalId
veterinarianId

Durante o load(), os animais e veterinários são carregados primeiro. Depois, ao carregar os serviços e prescrições, os IDs guardados no ficheiro são usados para procurar os objetos correspondentes nos containers.

Exemplo simplificado:

```
Animal* animal = clinic.getAnimalContainer().get(animalId);  
Veterinarian* veterinarian = clinic.getVeterinarianContainer().get(veterinarianId);
```

Figura 18-Reconstrução de apontadores no carregamento dos dados

Depois de encontrar os objetos, o Service ou a Prescription é reconstruído com os apontadores corretos.

Desta forma, o sistema mantém relações orientadas a objetos em memória, mas usa IDs no ficheiro binário para garantir que os dados podem ser guardados e carregados de forma segura.

7. Funcionalidades Implementadas

A aplicação implementa as principais funcionalidades necessárias para a gestão básica de uma clínica veterinária. As funcionalidades estão organizadas por menus, correspondendo às principais entidades do sistema: animais, veterinários, serviços e prescrições.

7.1 Gestão de Animais

A gestão de animais permite registar novos animais, listar todos os animais existentes e consultar um animal específico através do seu ID.

No registo de um animal, o utilizador introduz dados como o nome, espécie, raça, peso e idade. O ID do animal é gerado automaticamente pelo sistema, através do container correspondente.

As principais operações implementadas são:

- registar animal;
- listar animais;
- consultar animal por ID;
- editar animal;
- remover animal, quando aplicável.

7.2 Gestão de Veterinários

A gestão de veterinários permite registar e listar veterinários da clínica. Cada veterinário possui informação como nome, idade e especialidade.

O ID do veterinário também é gerado automaticamente, evitando que o utilizador tenha de introduzir manualmente identificadores.

As principais operações implementadas são:

- registar veterinário;
- listar veterinários;
- editar veterinário;
- remover veterinário, quando aplicável;
- consultar serviços associados a um veterinário.

7.3 Gestão de Serviços

A gestão de serviços permite criar serviços associados a um animal e a um veterinário. Cada serviço possui informação como tipo, custo, data e hora.

Ao criar um serviço, o utilizador introduz o ID do animal e o ID do veterinário. O `ServiceService` valida se estes objetos existem nos respetivos containers. Se algum dos objetos não existir, é lançada uma `DataConsistencyException`.

No Model, o serviço não guarda apenas os IDs introduzidos pelo utilizador. Depois da validação, o serviço é criado com apontadores para o Animal e para o Veterinarian associados.

As principais operações implementadas são:

- registar serviço;
- listar serviços;
- consultar serviços de um veterinário;
- editar serviço;
- remover serviço.

7.4 Gestão de Prescrições

A gestão de prescrições permite emitir prescrições associadas a um animal e a um veterinário. Cada prescrição contém informação sobre medicamento, quantidade e duração.

Tal como nos serviços, o utilizador introduz os IDs do animal e do veterinário. O PrescriptionService valida se estes objetos existem antes de criar a prescrição. No Model, a prescrição fica associada diretamente ao Animal e ao Veterinarian através de apontadores.

As principais operações implementadas são:

- emitir prescrição;
- listar prescrições;
- consultar prescrições associadas a um animal;
- editar prescrição;
- remover prescrição.

7.5 Consultas e Listagens

Além das operações de criação, edição e remoção, a aplicação permite consultar e listar informação relevante para o funcionamento da clínica.

As principais consultas implementadas são:

- listar todos os animais;
- listar todos os veterinários;
- listar todos os serviços;
- listar todas as prescrições;
- consultar animal por ID;
- consultar serviços associados a um veterinário;
- consultar prescrições associadas a um animal.

Estas consultas usam os containers da Clinic para procurar e devolver os dados necessários. Quando os dados são apresentados ao utilizador, os objetos do Model são convertidos para DTOs de saída através dos Mappers, evitando expor diretamente as entidades internas à camada de apresentação.

8. Tratamento de Erros e Validações

O tratamento de erros da aplicação é feito através de exceptions próprias. Esta abordagem permite representar diferentes tipos de erro de forma mais clara e organizada, evitando depender apenas de valores especiais de retorno.

As exceptions são usadas principalmente nos Models, Containers e Services, dependendo do tipo de erro. Quando ocorre uma situação inválida, é lançada uma exception, que pode depois ser tratada no Controller para apresentar uma mensagem ao utilizador.

As principais exceptions implementadas são:

- InvalidDataException
- NoDataException
- DuplicatedDataException
- DataConsistencyException

8.1 InvalidDataException

A `InvalidDataException` é usada quando os dados introduzidos ou atribuídos a um objeto são inválidos.

Esta exception pode ser usada, por exemplo, quando:

- o nome de um animal ou veterinário está vazio;
- a idade é negativa;
- o peso de um animal é inválido;
- a data ou hora de um serviço é inválida;
- o custo de um serviço é negativo.

Exemplo de utilização:

```
if (age < 0) {  
    throw InvalidDataException("Idade de Animal não pode ser negativa.");  
}
```

Figura 19-Exemplo de validação com `InvalidDataException`

Esta validação ajuda a impedir que objetos do sistema sejam criados ou modificados com dados incorretos.

8.2 NoDataException

A `NoDataException` é usada quando uma operação tenta aceder a um objeto que não existe.

Um exemplo é a consulta de um animal por ID. Se o animal não existir no container, o sistema lança uma `NoDataException`.

Exemplo:

```
Animal* animal = clinic.getAnimalContainer().get(id);  
  
if (animal == nullptr) {  
    throw NoDataException("Animal não encontrado.");  
}
```

Figura 20-Exemplo de utilização de `NoDataException`

Esta exception é útil para diferenciar situações em que os dados pedidos simplesmente não existem.

8.3 DuplicatedDataException

A DuplicatedDataException é usada para representar situações em que se tenta inserir ou registar dados duplicados.

No contexto da aplicação, esta exception pode ser usada nos containers para impedir a adição de objetos com o mesmo ID ou com informação que deva ser única.

Exemplo simplificado:

```
if (get(animal.getId()) != nullptr)
    throw DuplicatedDataException("Animal já existe.");
```

Figura 21-Exemplo de utilização de DuplicatedDataException

Esta validação contribui para manter os dados consistentes e evitar registos repetidos indevidamente.

8.4 DataConsistencyException

A DataConsistencyException é usada quando existe uma inconsistência entre objetos relacionados.

Um exemplo importante ocorre ao criar um serviço ou uma prescrição. Antes de criar o objeto, o sistema verifica se o animal e o veterinário indicados pelo utilizador existem. Caso contrário, a operação não deve ser concluída.

Exemplo:

```
Animal* animal = clinic.getAnimalContainer().get(dto.animalId);
Veterinarian* veterinarian = clinic.getVeterinarianContainer().get(dto.veterinarianId);

if (animal == nullptr || veterinarian == nullptr) {
    throw DataConsistencyException("Animal ou Veterinário não existe.");
}
```

Figura 22-Exemplo de utilização de DataConsistencyException

Esta exception também é útil em operações de remoção, por exemplo para impedir que um animal ou veterinário seja removido enquanto ainda estiver associado a serviços ou prescrições.

9. Testes

Para validar o funcionamento da aplicação, foi criado um projeto de testes separado, usando a framework Google Test. A organização deste projeto foi inspirada no tester da aplicação de exemplo *School Application*.

Os testes têm como objetivo verificar se as principais classes do sistema funcionam corretamente de forma isolada, reduzindo o risco de erros quando são feitas alterações ao código.

Atualmente, o tester inclui testes para:

- Models;
- Containers;
- Services;
- Repository;
- Exceptions;
- classes auxiliares, como Date e Time.

9.1 Estrutura do Projeto de Testes

O projeto de testes encontra-se separado do projeto principal da aplicação. Esta separação permite testar as classes do sistema sem executar diretamente a interface de consola.

A estrutura do tester segue uma organização simples, com ficheiros de teste separados por classe ou área funcional. Por exemplo:

`animal_test.cpp`

`animal_container_test.cpp`

`veterinarian_test.cpp`

veterinarian_container_test.cpp
service_test.cpp
service_container_test.cpp
prescription_test.cpp
prescription_container_test.cpp
clinic_repository_binary_test.cpp

O ficheiro main.cpp do tester inclui os ficheiros de teste e executa todos os testes através do Google Test:

```
int main(int argc, char* argv[]) {  
    ::testing::InitGoogleTest(&argc, argv);  
    return RUN_ALL_TESTS();  
}
```

Figura 23-Execução dos testes com Google Test

9.2 Testes ao Model e Containers

Foram criados testes para verificar o comportamento das principais classes do Model, como Animal, Veterinarian, Service, Prescription, Date e Time.

Estes testes verificam, por exemplo:

- criação correta dos objetos;
- valores devolvidos pelos getters;
- funcionamento de operadores, quando aplicável;
- validação de datas e horas inválidas;
- relações por apontadores em Service e Prescription.

Também foram testados os containers principais:

- AnimalContainer;
- VeterinarianContainer;
- ServiceContainer;
- PrescriptionContainer.

Nestes testes são verificadas operações como:

- adicionar objetos;
- procurar objetos por ID;

- devolver todos os objetos;
- gerar automaticamente o próximo ID.

9.3 Testes aos Services

Foram adicionados testes aos Services para validar a lógica principal da aplicação.

Por exemplo, no AnimalService foram testadas operações como adicionar um animal e consultar um animal por ID. Nos serviços relacionados com Service e Prescription, foram testadas situações válidas e inválidas.

Exemplos de situações testadas:

- adicionar um serviço válido;
- tentar adicionar um serviço com animal inexistente;
- tentar adicionar um serviço com veterinário inexistente;
- consultar serviços de um veterinário;
- adicionar uma prescrição válida;
- tentar adicionar uma prescrição com animal ou veterinário inexistente;
- consultar prescrições de um animal.

Estes testes são importantes porque os Services concentram grande parte da lógica da aplicação e das validações entre entidades.

9.4 Testes ao Repository

Também foram criados testes para o ClinicRepositoryBinary, responsável pela persistência dos dados em ficheiro binário.

Os testes ao Repository verificam se os dados são guardados e carregados corretamente entre diferentes instâncias do repository.

Foram testados cenários como:

- guardar e carregar animais e veterinários;
- guardar e carregar serviços;

- guardar e carregar prescrições;
- reconstruir corretamente os apontadores de Service e Prescription após o carregamento;
- carregar um ficheiro inexistente sem causar erro.

Um aspeto importante destes testes é confirmar que, depois do load(), os serviços e prescrições continuam associados aos animais e veterinários corretos.

9.5 Resultado dos Testes

Após a implementação dos testes, o projeto de testes executa com sucesso:

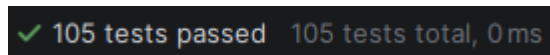


Figura 24-Resultado dos testes automáticos

Este resultado indica que os principais componentes da aplicação estão a funcionar corretamente de acordo com os cenários testados.

Além dos testes automáticos, também foram realizados testes manuais à aplicação, principalmente para verificar a persistência binária. O teste manual consistiu em criar animais, veterinários, serviços e prescrições, fechar a aplicação, abrir novamente e confirmar que os dados continuavam disponíveis.

10. Diagramas

Nesta secção são apresentados os principais diagramas usados para representar a aplicação. Estes diagramas ajudam a compreender a estrutura geral do sistema, o modelo de domínio e a organização das funcionalidades.

10.1 Diagrama de Arquitetura

O diagrama de arquitetura apresenta a organização geral da aplicação em diferentes camadas e módulos. A aplicação encontra-se organizada em views, controllers, services, repository, model, DTOs, mappers e exceptions.

A figura não representa todos os detalhes internos da implementação, mas mostra a

separação principal de responsabilidades dentro do projeto. Na prática, o `main.cpp` inicia a aplicação, o Controller coordena o fluxo, as Views tratam da interação com o utilizador, os Services executam a lógica da aplicação e o Repository trata da persistência dos dados.

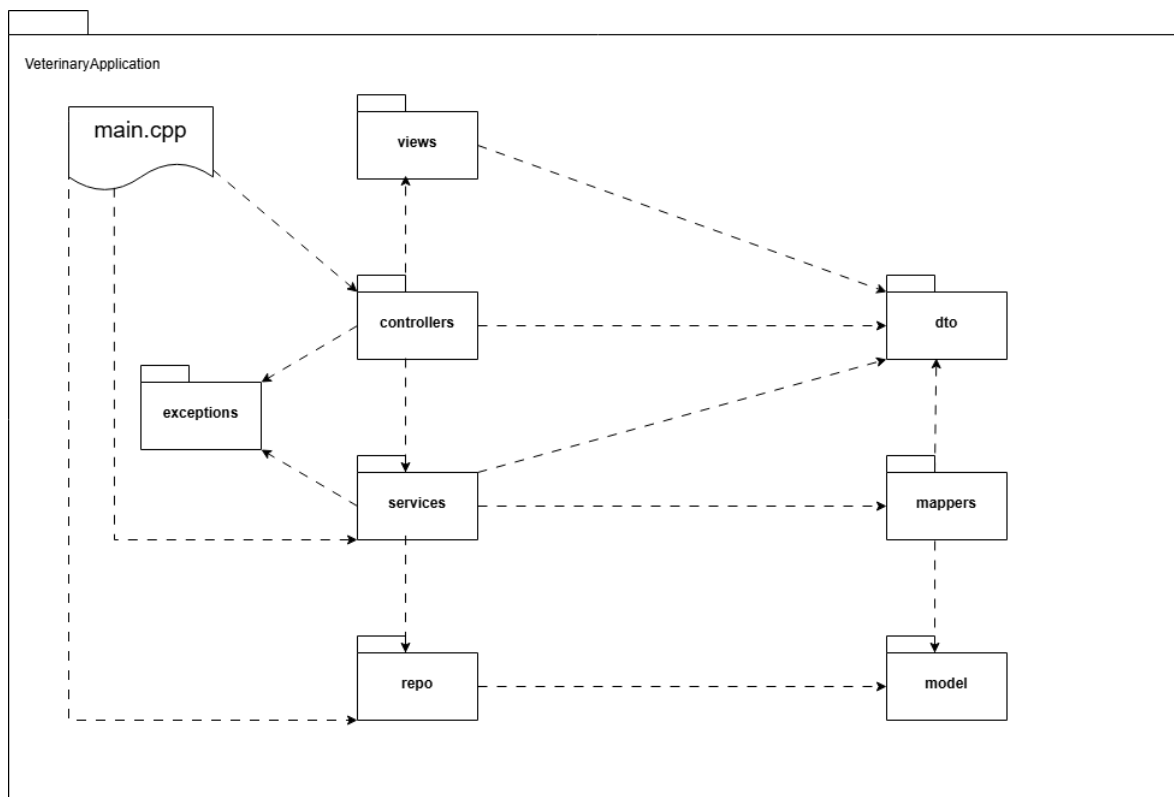


Figura 25-Diagrama de arquitetura da aplicação

10.2 Diagramas de Classes

Os diagramas de classes permitem representar a estrutura interna da aplicação, mostrando as classes existentes, os seus atributos, métodos e relações.

Ao longo das iterações do projeto foram desenvolvidos diagramas de classes para representar diferentes partes da aplicação, incluindo o Model, Views, Controller, Services, DTOs, Mappers, Repository, Containers e Exceptions.

No entanto, devido à dimensão do projeto e ao número de classes existentes, optou-se por não incluir todos os diagramas técnicos detalhados neste relatório. Em vez disso, foram apresentados os diagramas mais relevantes para a compreensão geral da aplicação, como o modelo de domínio, o diagrama de arquitetura e o diagrama do Repository.

Na iteração final foram feitas as correções necessárias para alinhar os diagramas com a implementação atual do projeto, nomeadamente ao nível das relações entre entidades, uso de apontadores no Model e separação da persistência através do Repository.

Os diagramas técnicos completos encontram-se disponíveis na documentação das respetivas iterações do projeto.

10.3 Diagrama do Repository

O diagrama do Repository representa a forma como a persistência foi separada da restante lógica da aplicação. A interface `IClinicRepository` define as operações principais do repository, enquanto as classes `ClinicRepositoryMemory` e `ClinicRepositoryBinary` representam duas implementações possíveis.

No contexto deste projeto, a implementação usada foi o `ClinicRepositoryBinary`, responsável por disponibilizar a instância de `Clinic` à aplicação e por guardar/carregar os dados no ficheiro binário `clinic.dat`.

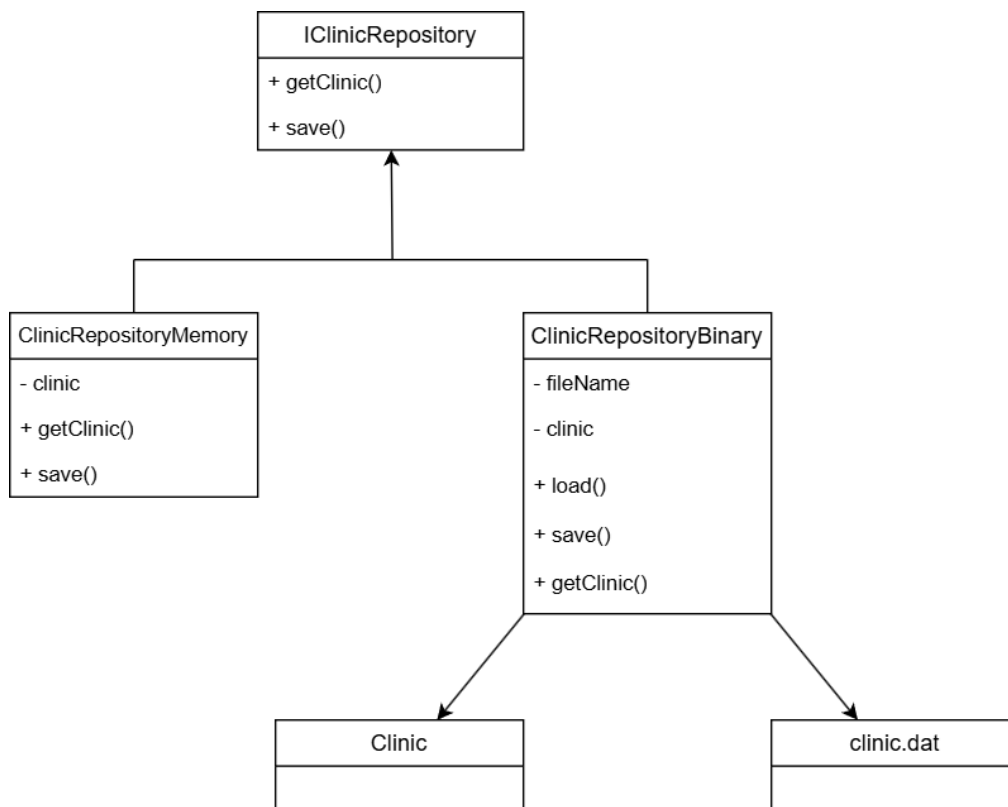


Figura 26-Diagrama do Repository

O diagrama mostra que o `ClinicRepositoryBinary` implementa a interface `IClinicRepository`, disponibiliza a `Clinic` à aplicação e trata da persistência dos dados no ficheiro `clinic.dat`.

10.4 Diagramas de Casos de Uso

Os diagramas de casos de uso representam as principais interações do utilizador com a aplicação. Estes diagramas foram desenvolvidos ao longo das iterações do projeto, acompanhando a evolução das funcionalidades previstas e implementadas.

Nas primeiras iterações foram definidos os casos de uso principais, como registar animais, registar veterinários, criar serviços, emitir prescrições e consultar informação. Na iteração final foram realizadas as correções necessárias para alinhar os diagramas com a implementação atual da aplicação.

Os diagramas detalhados das iterações anteriores encontram-se disponíveis nas respetivas pastas da documentação do projeto. No relatório é apresentado um diagrama geral de casos de uso, com o objetivo de resumir as funcionalidades principais do sistema.

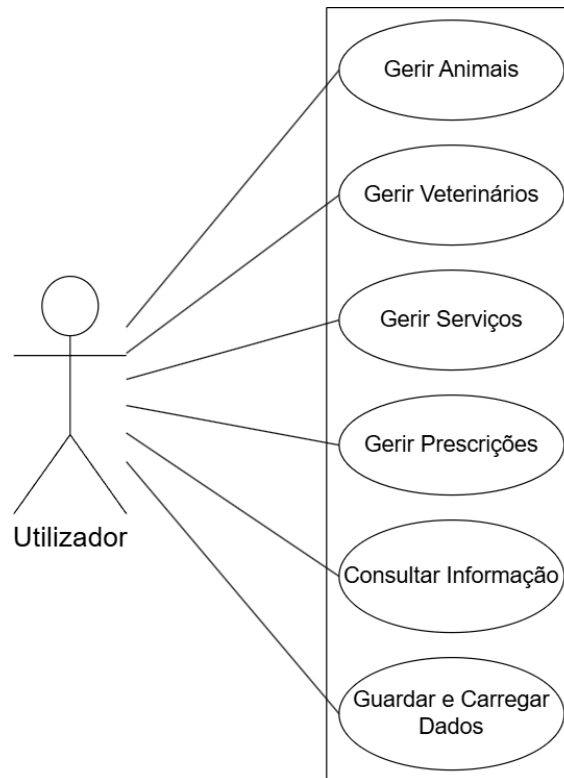


Figura 27-Diagrama geral de casos de uso

11. Limitações e Trabalho Futuro

Apesar de o projeto já implementar as principais funcionalidades previstas, ainda existem algumas limitações e melhorias possíveis.

Uma das limitações atuais é o facto de a aplicação ser baseada apenas em consola. Embora esta abordagem seja suficiente para o objetivo da unidade curricular, uma interface gráfica poderia tornar a utilização mais intuitiva e próxima de uma aplicação real.

Outra limitação está relacionada com algumas validações, que ainda podem ser melhoradas. O sistema já utiliza exceptions próprias para tratar erros e inconsistências, mas algumas regras de validação podem ser reforçadas, como a validação mais detalhada dos dados introduzidos pelo utilizador ou a verificação da compatibilidade entre o tipo de serviço e a especialidade do veterinário.

As operações de edição e remoção também podem ser melhoradas ou expandidas. Em particular, a remoção de entidades como animais e veterinários exige cuidado adicional, uma vez que estes podem estar associados a serviços ou prescrições.

Nestes casos, é necessário garantir que não são criadas inconsistências ou apontadores inválidos no sistema.

Como trabalho futuro, podem ser consideradas as seguintes melhorias:

- melhorar as validações dos dados introduzidos;
- validar serviços de acordo com a especialidade do veterinário;
- completar ou melhorar operações de edição e remoção;
- adicionar mais testes automáticos para as funcionalidades futuras;
- melhorar as mensagens apresentadas ao utilizador;
- melhorar a interface da aplicação;
- atualizar os diagramas finais de acordo com a implementação.

12. Conclusão

O desenvolvimento deste projeto permitiu aplicar vários conceitos importantes de Programação Orientada a Objetos e organização de software em C++. A aplicação foi estruturada em diferentes camadas, separando a interface, o controlo, a lógica da aplicação, o modelo de domínio, a persistência e os testes.

Ao longo do projeto foram implementadas funcionalidades de gestão de animais, veterinários, serviços e prescrições. Também foi implementada persistência em ficheiro binário através de uma camada Repository, permitindo guardar e carregar os dados entre execuções da aplicação.

Um dos aspetos mais relevantes da implementação foi a representação das relações no Model através de apontadores para objetos, em vez de guardar apenas IDs. Para a persistência, estas relações são guardadas através de IDs no ficheiro binário e reconstruídas no carregamento dos dados.

O projeto também inclui tratamento de erros através de exceptions próprias e testes automáticos com Google Test, o que ajudou a verificar o funcionamento das classes principais, containers, services e repository.

De forma geral, o projeto cumpriu os principais objetivos definidos, servindo como exercício prático de aplicação de conceitos de desenvolvimento de software, arquitetura em camadas, persistência, tratamento de erros e testes.

13. Referências

Durante o desenvolvimento do projeto foram usados como referência os seguintes recursos:

- Material disponibilizado pelo docente da unidade curricular;
- Aplicação de exemplo *School Application*;
- Tester de exemplo *SchoolApplicationTester*;
- Documentação oficial e exemplos de C++ sobre ficheiros, `fstream`, `ifstream` e `ofstream`;
- Documentação e exemplos de utilização do Google Test;
- Repositório/template de README:
<https://github.com/othneildrew/Best-README-Template>

Também foram consultados recursos gerais sobre Programação Orientada a Objetos, organização de projetos em C++ e persistência de dados em ficheiros binários.